



NOTAS DE REFLEXIÓN · QA & AI ENGINEERING

Testing no funcional en sistemas de IA

Rendimiento, escalabilidad, confiabilidad y robustez adversarial:
qué cambia cuando el sistema bajo prueba responde con una
probabilidad en lugar de una certeza.

CONTENIDO

- 01 Introducción al testing no funcional
- 02 Rendimiento y escalabilidad
- 03 Confiabilidad y resiliencia
- 04 Robustez ante inputs adversos
- 05 Exploración práctica y reflexión final

1. Introducción al testing no funcional

Diferencias entre testing funcional y no funcional en un modelo de IA

En software tradicional la frontera es nítida: lo funcional pregunta “¿*hace lo que debe?*” y lo no funcional “¿*cómo de bien lo hace?*”. En IA esa frontera se vuelve porosa, porque la calidad de la respuesta **es** parte del comportamiento observable.

DIMENSIÓN	TESTING FUNCIONAL	TESTING NO FUNCIONAL
Pregunta central	¿El sistema produce la salida esperada?	¿En qué condiciones lo sigue haciendo, y a qué coste?
Oráculo	Aserción exacta o semántica sobre la respuesta	Distribución de métricas frente a umbrales (SLO)
Naturaleza del resultado	Pasa / falla	Percentiles, tasas, tendencias
Ejecución típica	Determinista, repetible	Estadística, requiere N ejecuciones
Ejemplo	“Ante la consulta X, el chatbot debe derivar a un humano”	“El 95 % de las respuestas llega en menos de 3 s con 500 usuarios concurrentes”

Tres diferencias que me parecen las más relevantes:

- **El oráculo deja de ser binario.** En un API REST clásico, `200 OK` con el JSON esperado es la verdad. En un modelo generativo, dos respuestas distintas pueden ser ambas correctas, y una tercera puede ser plausible pero falsa. Eso obliga a medir con métricas agregadas (tasa de acierto, similitud semántica, tasa de alucinación) en lugar de aserciones exactas, y difumina la línea entre “funcional” y “de calidad”.
- **El no determinismo convierte cualquier prueba en muestral.** Una sola ejecución no dice nada. Hay que repetir N veces y razonar sobre la distribución, algo mucho más cercano al testing de rendimiento que al funcional.
- **El coste es una métrica de primer nivel.** En software tradicional el coste por transacción rara vez aparece en una prueba. En IA, tokens consumidos y euros por conversación son tan críticos como la latencia, y pueden hacer inviable en producción un sistema que funciona perfectamente en la demo.

Aspectos no funcionales más críticos en sistemas de IA

Ordenados por lo que suele doler antes en producción:

1. **Latencia percibida.** No solo el tiempo total: en interfaces conversacionales el *time to first token* (TTFT) domina la percepción de fluidez. Un sistema con 8 s de respuesta total pero 400 ms hasta el primer token se siente mucho más rápido que otro con 5 s totales y 4 s de silencio inicial.
2. **Coste por interacción.** Métrica no funcional específica de IA. Escala linealmente con el uso y con la longitud del contexto, no con la infraestructura.
3. **Estabilidad y consistencia de las salidas.** ¿La misma pregunta devuelve respuestas equivalentes a lo largo del tiempo? Aquí entran temperatura, versionado de modelo y cambios silenciosos del proveedor.
4. **Robustez ante entradas fuera de distribución.** Lo que el modelo no vio en entrenamiento.
5. **Escalabilidad.** Comportamiento bajo carga concurrente, con la particularidad de que el cuello de botella es cómputo (GPU) o cuota del proveedor, no CPU/memoria convencionales.
6. **Seguridad específica de IA.** Prompt injection, fuga de datos del contexto, jailbreaking. Es el ámbito donde testing no funcional y adversarial se solapan más.
7. **Observabilidad y trazabilidad.** Si no se pueden reconstruir prompt, contexto, versión de modelo y respuesta, no hay diagnóstico posible.
8. **Sesgo y equidad.** Rendimiento comparable entre subgrupos de población. Es un requisito no funcional con

implicaciones regulatorias (AI Act).

Por qué importa la robustez ante inputs inesperados

Porque un modelo **nunca dice “no sé cómo procesar esto”**. Un parser tradicional falla con una excepción visible; un LLM ante una entrada absurda genera con toda naturalidad una respuesta absurda, con el mismo tono de confianza que usa cuando acierta. El fallo no es ruidoso, es silencioso, y llega al usuario final indistinguible de una respuesta buena.

A esto se suma que en producción los inputs reales son mucho peores que los de los datasets de prueba: faltas de ortografía, mezcla de idiomas, copy-paste con formato roto, campos vacíos, PDFs escaneados torcidos, y usuarios que directamente intentan romper el sistema. Si solo se prueba el camino feliz, se está midiendo el modelo en un régimen que no existe fuera del laboratorio.

2. Rendimiento y escalabilidad

Cómo medir latencia y tiempo de respuesta bajo diferentes cargas

Descomponer la latencia antes que medirla en bloque. Para un sistema conversacional con RAG:

```
latencia_total =  
    tiempo_deCola  
+ preprocesado / embedding de la consulta  
+ búsqueda en el vector store (recuperación)  
+ construcción del prompt  
+ TTFT (primer token del modelo)  
+ generación de los tokens restantes  
+ postprocesado / guardarrailes / formateo
```

Medir cada tramo por separado es lo que permite decir *dónde* está el problema. Es habitual descubrir que el 40 % del tiempo se va en la recuperación y no en el modelo, lo que cambia por completo la decisión de optimización.

Métricas concretas a instrumentar:

MÉTRICA	QUÉ MIDE	POR QUÉ IMPORTA
TTFT	Tiempo hasta el primer token	Percepción de respuesta inmediata
TPOT / ITL	Tiempo por token de salida (inter-token latency)	Fluidez de la escritura en streaming
Latencia end-to-end p50 / p95 / p99	Distribución completa	La media miente; el p99 es lo que sufre el usuario descontento
Throughput (req/s y tokens/s)	Capacidad agregada	Dimensionamiento
Tokens de entrada / salida por petición	Consumo	Coste y saturación de contexto
Tasa de error y de rate-limit (429)	Fiabilidad bajo carga	Punto de ruptura real
Coste por 1 000 peticiones	€ por unidad de negocio	Viabilidad económica

Nota importante sobre percentiles: **nunca se promedian entre ejecuciones ni entre nodos**. Un p95 de p95 no significa nada. Hay que conservar los histogramas y calcular el percentil sobre la muestra completa.

Diseño de las pruebas de carga:

- **Carga base (smoke)**. 1 usuario, N repeticiones. Establece el suelo de latencia y la variabilidad inherente del modelo. Sin esta línea base no hay con qué comparar.
- **Carga escalonada (ramp-up)**. Incrementos progresivos de concurrencia (1 → 5 → 10 → 25 → 50 → 100) manteniendo cada escalón varios minutos. Se busca el punto de inflexión: la concurrencia a partir de la cual la latencia crece más

rápido que linealmente mientras el throughput se aplan. Ese punto es la capacidad real.

- **Prueba de resistencia (soak).** Carga sostenida y moderada durante horas. Detecta fugas de memoria, degradación de caché, crecimiento del vector store y acumulación de contexto en sesiones largas.
- **Prueba de pico (spike).** Salto brusco de carga. Verifica el comportamiento del autoescalado y, sobre todo, si el sistema **degrada con dignidad** (cola con mensaje de espera, modelo más pequeño de respaldo, respuesta cacheada) o simplemente devuelve 500.
- **Prueba de estrés.** Empujar hasta romper, deliberadamente, para conocer el límite y verificar que la recuperación posterior es limpia.

Detalles propios de IA que cambian el diseño de la prueba:

- El *payload* importa tanto como el número de usuarios. 100 peticiones con 200 tokens de contexto no se parecen en nada a 100 peticiones con 20 000. El plan de carga debe usar una **distribución de longitudes de prompt representativa del tráfico real**, no un prompt fijo.
- Si hay caché semántica, un script que repite siempre la misma consulta mide la caché, no el modelo. Hace falta variedad léxica en los datos de prueba.
- Medir con *streaming* activado si así se sirve en producción; si no, TTFT no es observable.
- Los límites del proveedor (peticiones por minuto y **tokens** por minuto) suelen alcanzarse antes que cualquier límite técnico. Conviene probar contra un entorno con cuota equivalente a producción, y avisar al proveedor de que se va a lanzar una prueba de carga.
- El coste de una prueba de carga contra un modelo de pago es real. Hay que presupuestarla antes de lanzarla.

Implicaciones de un modelo grande en recursos y escalabilidad

Si el modelo es propio (self-hosted):

- La memoria de GPU necesaria $\approx$ (nº de parámetros $\times$ bytes por parámetro) + caché KV + activaciones. La caché KV crece con el número de secuencias concurrentes **y** con la longitud del contexto de cada una: es habitualmente el factor que limita la concurrencia real, más que el peso del modelo.
- Escalar no es “añadir réplicas”: cada réplica exige una GPU (o varias), con tiempos de arranque en frío de minutos y disponibilidad limitada. El autoescalado reactivo típico de microservicios no funciona bien aquí; hace falta sobreaprovisionar o mantener capacidad templada.
- Palancas de optimización: cuantización (con la contrapartida de posible pérdida de calidad, que hay que medir), *batching* continuo, *paged attention*, destilación a un modelo más pequeño, enrutado por complejidad (modelo pequeño para consultas simples, grande solo cuando hace falta).

Si el modelo es de un tercero vía API:

- La escalabilidad se compra, pero está limitada por cuotas contractuales y por la salud del proveedor, que es una dependencia externa fuera de control.
- El coste crece linealmente con el uso, sin economías de escala. Un aumento de tráfico de 10 $\times$ multiplica la factura por 10.
- Riesgo específico: **el modelo puede cambiar bajo los pies del sistema**. Actualizaciones, deprecaciones o ajustes del proveedor alteran latencia y calidad sin ningún cambio en el código propio. Fijar la versión del modelo cuando el proveedor lo permita, y monitorizar como si fuera un componente externo volátil.

Palancas transversales de escalabilidad: caché semántica de respuestas frecuentes, procesamiento asíncrono con cola para lo que no requiera respuesta inmediata, recorte inteligente del contexto (el contexto largo cuesta latencia y dinero), y respuestas precalculadas para el conjunto de consultas más habituales.

Simulación de alto tráfico y experiencia de usuario

Lo que hace realista una simulación no es el número de hilos, sino la fidelidad del perfil de tráfico: mezcla de tipos de consulta, distribución horaria (los picos suelen ser predecibles), sesiones con historial acumulado frente a sesiones

nuevas, y tiempos de reflexión entre turnos.

Y el criterio de aceptación debería expresarse en términos de experiencia, no de infraestructura. Por ejemplo:

Con 200 usuarios concurrentes, el 95 % de las respuestas debe empezar a mostrarse en menos de 1,5 s, la tasa de error debe mantenerse por debajo del 0,5 %, y ninguna petición debe quedar sin respuesta más de 30 s sin comunicar al usuario que sigue en proceso.

Un matiz que me parece central: bajo carga, un sistema de IA rara vez “se cae”. Lo que hace es **volverse lento y, a veces, peor**. Si hay degradación automática a un modelo más pequeño o a un contexto recortado, la calidad de la respuesta cae sin que se dispare ninguna alarma técnica. Por eso la prueba de carga debe medir también **calidad bajo carga**, no solo tiempos: ejecutar el set de evaluación funcional mientras la carga está activa y comparar resultados contra la ejecución en reposo.

3. Confiabilidad y resiliencia

Qué significa que un modelo sea confiable en producción

Confiable no es “acierta siempre” — ningún modelo lo hace. Confiable es que su comportamiento sea **predecible y acotado**:

- **Consistencia:** entradas equivalentes producen salidas equivalentes, dentro de un margen conocido y aceptado.
- **Estabilidad temporal:** la calidad medida hoy es comparable a la de hace tres meses, o si ha cambiado, se sabe.
- **Fallo explícito antes que fallo silencioso:** ante baja confianza o falta de contexto, el sistema dice que no puede responder o deriva a un humano, en lugar de inventar.
- **Contención del blast radius:** un fallo del modelo no arrastra al resto del sistema. Timeouts, circuit breakers y camino de respaldo definido.
- **Reproducibilidad:** cualquier respuesta puede reconstruirse a partir de los datos registrados (prompt, contexto recuperado, versión de modelo, parámetros).

Cómo detectar errores silenciosos y resultados inconsistentes

Es el problema más difícil del testing no funcional en IA, porque por definición no hay excepción que capturar. Estrategias que se complementan:

Golden dataset con ejecución periódica. Un conjunto curado de entradas con salidas de referencia, ejecutado de forma programada contra producción o un espejo. No se compara texto literal, sino similitud semántica, presencia de hechos clave y ausencia de contenido prohibido. La señal útil es la **tendencia**: una caída sostenida de la puntuación media indica deriva.

Autoconsistencia. Lanzar la misma consulta N veces (o con parafraseos equivalentes) y medir la dispersión de las respuestas. Alta varianza en preguntas que deberían tener respuesta única es un indicador temprano de inestabilidad.

Chequeos invariantes. Propiedades que deben cumplirse siempre, verificables sin conocer la respuesta correcta: el formato de salida es JSON válido, las cifras citadas aparecen en el contexto recuperado, no se mencionan entidades ausentes del contexto, el idioma de la respuesta coincide con el de la pregunta, la longitud está en rango. Es *property-based testing* aplicado a IA, y detecta una fracción sorprendente de los fallos reales.

Verificación cruzada (LLM-as-a-judge). Un segundo modelo evalúa la respuesta del primero contra el contexto. Útil y escalable, pero con la advertencia de que el juez también se equivoca y también deriva: hay que calibrarlo periódicamente contra etiquetas humanas y tratar su puntuación como una señal, no como la verdad.

Detección de deriva en las entradas. Monitorizar la distribución de los prompts de entrada (longitud, idioma, temas, embeddings). Cuando el tráfico real se aleja de aquel para el que se validó el sistema, la calidad va a caer aunque el

modelo no haya cambiado. Es una alerta que llega *antes* del incidente.

Señales implícitas del usuario. Reformulaciones de la misma pregunta, abandono de la conversación, escalado a agente humano, pulgar hacia abajo. Son un detector de fallos gratuito y en tiempo real, y suelen correlacionar mejor con la calidad percibida que cualquier métrica offline.

Monitoring y alertas

Qué instrumentar en cada petición: identificador de traza, versión del modelo y del prompt, latencia por tramo, tokens de entrada/salida, coste, activaciones de guardarrailes, puntuación de recuperación, feedback del usuario. Con eso se puede reconstruir cualquier incidente; sin ello, el diagnóstico es adivinación.

Alertas técnicas (umbral): p95 de latencia por encima del SLO, tasa de error o de 429 al alza, caída del throughput, coste por hora fuera de presupuesto, disponibilidad del proveedor.

Alertas de calidad (tendencia, ventana móvil): caída de la puntuación del golden dataset, aumento de la tasa de “no sé responder”, aumento de activaciones de guardarrailes, subida de escalados a humano, desviación de la distribución de embeddings de entrada.

Distinción práctica: las técnicas se alertan al instante; las de calidad casi siempre por tendencia sobre ventana, porque el ruido puntual es alto y una alerta por cada respuesta mala genera fatiga y termina ignorándose.

Y un control económico específico de IA: límites duros de gasto y de tokens por sesión. Un bucle de agentes mal acotado o un ataque de amplificación de contexto puede generar una factura muy alta en minutos. Es un fallo no funcional sin equivalente claro en software tradicional.

Sistema que interactúa con clientes en tiempo real: pruebas imprescindibles antes de lanzar

Priorizadas como bloqueantes de salida a producción:

1. **Prueba de carga al pico esperado x2**, con perfil de tráfico realista, midiendo latencia, errores **y calidad simultáneamente**.
2. **Prueba de degradación:** ¿qué ocurre cuando el proveedor del modelo responde lento, devuelve 429 o cae? Debe existir y estar probado un camino de respaldo (modelo alternativo, respuesta cacheada, derivación a humano) con mensaje claro al usuario.
3. **Prueba de seguridad de prompts:** batería de prompt injection, intentos de extracción del system prompt, jailbreaking y solicitudes de contenido prohibido. Con métrica de tasa de bypass, no con un “lo intentamos y aguantó”.
4. **Prueba de fuga de datos:** verificar que el modelo no expone datos de otros usuarios, credenciales, ni información del contexto de sesiones ajenas. Crítico si hay multitenencia.
5. **Prueba de robustez de entradas:** ruido, idiomas mezclados, entradas vacías o desmesuradas, caracteres de control, emojis, texto sin sentido.
6. **Validación de guardarrailes y de la ruta de escalado a humano**, que es la red de seguridad de todo lo demás y suele ser lo menos probado.
7. **Verificación de observabilidad:** confirmar que ante un fallo se puede reconstruir qué pasó. Si no, cualquier incidente en producción será irresoluble.
8. **Prueba de coste bajo carga:** proyección de la factura al tráfico esperado, con los límites de gasto activos y verificados.

4. Robustez ante inputs adversos

Cómo afectan los inputs inesperados, incompletos o maliciosos

Inputs inesperados (fuera de la distribución de entrenamiento): el modelo extrapola. El resultado puede ser razonable o

completamente inventado, y desde fuera ambos casos son indistinguibles porque el tono de confianza es idéntico.

Inputs incompletos o ruidosos: degradación gradual y a menudo desigual. Faltas de ortografía o transcripciones automáticas defectuosas pueden hacer fallar la recuperación en un sistema RAG — si el embedding de la consulta se desplaza, se recuperan documentos irrelevantes y el modelo genera una respuesta perfectamente redactada sobre el contexto equivocado. El fallo ocurre antes del modelo, pero se manifiesta en su salida.

Inputs maliciosos: la superficie de ataque específica de IA. Prompt injection directa e indirecta (instrucciones ocultas en un documento que el sistema recupera y procesa como si fueran del usuario), extracción del system prompt, jailbreaking, envenenamiento de la base de conocimiento, y agotamiento de recursos por amplificación de contexto o bucles de razonamiento. La injection indirecta merece atención especial: en cuanto el sistema ingiere contenido externo (web, PDFs, correos, tickets), ese contenido es una entrada no confiable con capacidad de influir en el comportamiento del modelo.

Hay además un fallo estructural del propio paradigma: **el modelo no distingue de forma fiable entre instrucciones y datos**. Todo llega como texto en la misma ventana de contexto. Eso hace que la defensa no pueda depender solo del prompt del sistema, y tenga que apoyarse en controles fuera del modelo: validación de entrada, filtrado de salida, aislamiento del contenido no confiable y permisos mínimos en las herramientas que el modelo pueda invocar.

Relación entre testing adversarial y testing no funcional

Se solapan más de lo que la separación clásica sugiere. Ambos comparten la misma premisa —*el sistema se somete a condiciones distintas de las nominales para observar cómo se comporta*— y ambos producen métricas de distribución en lugar de veredictos binarios. La diferencia está en el origen de la presión: en el no funcional clásico es el volumen y el entorno; en el adversarial, una intención hostil.

En IA convergen de forma natural, porque un ataque adversarial es un problema de rendimiento y disponibilidad además de seguridad: una injection que provoca respuestas larguísimas es un ataque de coste y de latencia; un bucle de agentes inducido es una denegación de servicio; una entrada muy larga puede saturar el contexto y expulsar información legítima.

Estrategias combinadas que aumentan seguridad y estabilidad a la vez:

- **Fuzzing semántico bajo carga.** Generar variaciones automáticas de prompts (parafraseo, ruido tipográfico, cambio de idioma, inversión de intención) y lanzarlas dentro de la prueba de carga. Mide robustez y rendimiento en la misma ejecución, y a veces revela que la degradación de calidad solo aparece con concurrencia alta.
- **Red teaming continuo en el pipeline.** Una batería versionada de ataques conocidos, ejecutada en cada despliegue con una tasa de bypass como criterio de aceptación. Convierte la seguridad de IA en una métrica con umbral, no en una auditoría puntual.
- **Testing metamórfico.** Definir transformaciones que no deberían cambiar la respuesta (reformular una pregunta, cambiar el orden de los elementos de una lista, traducir y volver a traducir) y verificar que la salida se mantiene equivalente. No requiere conocer la respuesta correcta, lo que lo hace aplicable a escala, y detecta a la vez inestabilidad y vectores de manipulación.
- **Presupuestos de recursos como control de seguridad.** Límites de tokens por petición y por sesión, timeouts, tope de iteraciones en agentes, límite de gasto. Se diseñan como controles de rendimiento y funcionan además como contención de abuso.
- **Guardarrailes con métricas propias.** Los filtros de entrada y salida deben medirse como cualquier clasificador: falsos positivos (bloquear peticiones legítimas, que es un problema de usabilidad) y falsos negativos (dejar pasar lo peligroso). Y hay que medir la latencia que añaden, porque un guardarraíl caro puede duplicar el tiempo de respuesta.
- **Defensa en profundidad.** Validación de entrada, aislamiento del contenido no confiable, principio de mínimo privilegio en las herramientas del modelo, filtrado de salida y registro completo. Ninguna capa aguanta sola.

5. Exploración práctica

Ideas concretas para llevar esto a la práctica sin montar una plataforma entera:

Simulación de carga. Un script con concurrencia configurable (`asyncio` + `httpx` en Python, o k6/Locust si se quiere reporting) contra el endpoint del modelo, que registre por petición: timestamp, TTFT, latencia total, tokens de entrada y salida, código de estado. Con eso se calculan p50/p95/p99, throughput y coste. Ejecutarlo en escalones de concurrencia y graficar latencia y throughput frente a concurrencia: el punto donde el throughput se aplana mientras la latencia sube es la capacidad real del sistema.

Sensibilidad al ruido. Partir de un set de 30–50 consultas representativas y generar variantes degradadas de forma programática: erratas aleatorias (5 %, 10 %, 20 % de caracteres), eliminación de acentos y puntuación, truncado de la consulta, mezcla de idiomas, mayúsculas totales. Medir la degradación de la puntuación de calidad por nivel de ruido. La curva resultante indica cuánta suciedad tolera el sistema antes de romperse, que es exactamente el margen que hay en producción.

Consistencia. Lanzar cada consulta 10 veces y medir la similitud entre respuestas (embeddings + similitud coseno). Las consultas con dispersión alta son las candidatas a fallar de forma impredecible ante clientes.

Dashboards. Merece la pena revisar cómo estructuran sus métricas las plataformas de observabilidad de LLM (LangSmith, Langfuse, Arize Phoenix, o las capacidades de tracing de LLM en Datadog / Grafana + OpenTelemetry, cuyo estándar de convenciones semánticas para GenAI sigue evolucionando). Más allá de la herramienta concreta, el panel útil combina tres capas: **técnica** (latencia, errores, throughput), **económica** (tokens y coste por hora y por usuario) y **de calidad** (puntuación del golden set, feedback, activaciones de guardarrailes, tasa de escalado). Un dashboard que solo muestre la primera capa dará todo en verde mientras la calidad se desploma.

Reflexión final

Diferencias entre testear software tradicional y modelos de IA desde el enfoque no funcional

La diferencia de fondo es que en software tradicional **el sistema es correcto o incorrecto, y lo no funcional describe sus márgenes**; en IA el sistema es correcto *con una probabilidad*, y esa probabilidad se mueve con la carga, con el modelo, con los datos y con el tiempo. Lo no funcional deja de ser una capa alrededor de la funcionalidad y pasa a ser inseparable de ella.

De ahí se derivan cinco cambios prácticos:

1. **De aserciones a distribuciones.** El resultado de una prueba ya no es verde o rojo, sino una métrica con un umbral acordado. Hay que aprender a discutir con negocio en términos de percentiles y tasas aceptables.
2. **De regresión a deriva.** El software tradicional no empeora solo; un sistema de IA sí, porque el mundo cambia bajo él aunque el código esté congelado. Eso convierte la monitorización continua en parte del testing, no en algo posterior.
3. **De fallos ruidosos a fallos silenciosos.** No hay stack trace. El sistema responde con seguridad algo incorrecto, y detectarlo requiere una capa de evaluación explícita que en software clásico no existe.
4. **El coste entra en la ecuación.** Un requisito no funcional económico —euros por conversación— puede tumbar un proyecto que pasa todas las pruebas técnicas.
5. **Seguridad y no funcional se fusionan.** La superficie de ataque en lenguaje natural convierte cada prueba de robustez en una prueba de seguridad y viceversa.

Habilidades y mentalidad necesarias

Habilidades:

- **Estadística aplicada.** Percentiles, varianza, tamaño de muestra, significancia. Sin esto no se puede interpretar nada de lo anterior, y es probablemente la brecha más común en perfiles de QA que vienen del testing funcional.
- **Ingeniería de datos.** Construir y mantener datasets de evaluación representativos, versionados y con etiquetas fiables. El dataset es el activo de calidad más valioso y el más descuidado.

- **Observabilidad.** Instrumentación, trazas distribuidas, diseño de dashboards y de alertas que no generen fatiga.
- **Fundamentos de ML.** No hace falta entrenar modelos, pero sí entender embeddings, ventana de contexto, temperatura, caché KV y las diferencias entre RAG y fine-tuning, para saber dónde buscar la causa de un síntoma.
- **Seguridad aplicada a IA.** Conocer el catálogo de ataques (el OWASP Top 10 para aplicaciones LLM es un buen punto de partida) y saber traducirlo a casos de prueba ejecutables.
- **Automatización sólida.** Todo lo anterior debe correr en el pipeline, no en la máquina de alguien.

Mentalidad — que es lo que más cambia:

- **Probabilística en lugar de binaria.** Aceptar que un porcentaje de fallo es el estado normal, y que el trabajo consiste en conocerlo, acotarlo y decidir si es aceptable, no en eliminarlo.
- **Escéptica frente a lo elocuente.** Una respuesta bien redactada no es una respuesta correcta. La fluidez del lenguaje engaña al revisor humano de forma sistemática, y hay que desconfiar activamente de ella.
- **Continua, no de fase.** No hay un momento en el que un sistema de IA “está probado”. La calidad es una propiedad que se mantiene, no un hito que se alcanza.
- **Adversarial por defecto.** Preguntarse siempre cómo rompería alguien esto, no solo si funciona.
- **Económica.** Incorporar el coste como criterio de calidad de pleno derecho.
- **Humilde ante la incertidumbre.** Reconocer los límites del sistema y diseñar para que falle bien: derivar a un humano, decir “no lo sé”, degradar con transparencia. En IA, saber fallar es una característica de calidad, no una concesión.

En resumen: el testing no funcional en IA se parece menos a ejecutar un plan de pruebas y más a **operar un sistema de medición permanente** sobre algo que cambia por su cuenta. El perfil que hace falta está a medio camino entre el QA de rendimiento, el analista de datos y el ingeniero de fiabilidad.

#TestingNoFuncionalIA #AIQuality